

One Riri

There are **four** Riris in the estate today, three corpora, two embedding models and two incompatible ways of scoping a tenant — and the one person who actually waits for an answer, the customer, has no assistant at all. This is the plan to collapse that into **one brain with five contracts**, put her in front of every system including a lender's own, and let anyone train her on their JSON without ever letting that JSON invent a button.

PREPARED FOR Faith Birgen STATUS Sprint 0 built · 11 Sep 2026 READS WITH Blueprint v2 §6 · Tailnet Cutover
SCOPE 6 systems · 1 package · 1 corpus

01 What is actually running right now

Read before deciding anything. Each row below was read out of the source this morning, not off a diagram. The important column is the last one — because three of these four have a different answer to “what stops her making a number up”, and one of them has no answer because there is nothing there.

TABLE 1 · THE FOUR ASSISTANTS, AS BUILT

SURFACE	ENGINE & RETRIEVAL	CORPUS	LIVE DATA	WHAT HOLDS HER HONEST
BirgenAI Hub birgen-ai-frontend	Gemini 2.5 Flash · lib/rag.ts → Supabase pgvector riri_knowledge, match_riri_kb	11 JSON packs in src/data/kb/	None	PROMPT ONLY the persona and the “send them to movies.birgenai.com” rules are sentences in a system prompt
ServiceSuite LMS C# · AiIntelligence	Gemini embeds, Claude generates · match_riri_kb_entity with an entity_id bucket	ServiceSuiteKnowledge.Entries (system, entity 0) + per-lender JSON via SeedLenderKnowledge	Text-to-SQL (AskAnalytics) and a 7-tool Claude loop (ServiceSuiteAI) with AI-balance billing	PARTLY IN CODE DML regex, blocked-table list, TOP 500 — but tenant scope is a <i>substring test</i> for the entity id
Micro Eazy console connected- suite	Claude + Gemini · 25 modules under lib/riri/, deterministic router, metric catalogue	knowledge.ts (articles, EN+SW) + system-map.ts (65 screens) + catalog.ts	Prisma on the live book; the SQL relay for ServiceSuite books	IN CODE, TESTED rights, entitlements, server- resolved subject, drift test on the map, router pinned by a test
Micro Eazy app micro-eazy-app	—	—	—	NOTHING the customer writes into /api/portal/messages and waits for a human

The three things that will bite if they are not fixed first

- **A shadow copy of the retrieval code, disagreeing with the live one.**
CORRECTED 11 SEP This page first reported a live embedding mismatch. That was wrong, and the way it was wrong is the finding. `src/data/kb/` holds **committed** copies of `rag.ts`, `seed-kb.ts` and `route.ts` that name `text-embedding-004`; the real files under `src/lib/` and `src/scripts/` both use `gemini-embedding-001` at 768 and always did — as does the C# seeder. Nothing is broken. But that folder is not a scratch drawer: its `*.json` files *are* the live corpus, read by the seeder at `src/data/kb`, so live data and a stale shadow of the retrieval code sit in one directory. It is exactly where a person or a model looks when asked to read the knowledge base, and what is sitting there is a plausible, out-of-date account of how retrieval works. I read it first and believed it. The `.ts/.tsx` files there are referenced by nothing and are recoverable from git — delete them.
- **Two tenancy conventions.** The C# side scopes by `entity_id` `INTEGER` (0 = shared, N = lender). The console scopes by `orgId` `uuid`. A shared corpus cannot have two primary keys, and *member codes already collide* — `EntityId` 3003 is “Micromart Check-off” on one server and “Axe — Boresha” on the other (Tailnet Cutover, \$NEXT). Key the corpus on the same thing the Interchange will key members on.
- **The substring scope check.** `AIIntelligenceController` admits a generated query if the SQL merely *contains* the entity id as text. `EntityId = 300` passes the check for entity 3002 because “3002” appears in a date, a limit, an id. Ten unrelated companies share the .198 book separated by nothing but `EntityId`. This is the single highest-severity item on the page.

FOUND WHILE BUILDING SPRINT 0 — OUR OWN BOOK, NOT MICROMART'S

The console's tenant isolation is **not in force in this environment**, and every visible signal says it is. Loan, Borrower and the rest have rowsecurity AND forcerowsecurity true, the tenant_isolation policy is correct, all fourteen riri_* views are security_invoker=true, and readpath.ts correctly stamps app.org_id on its own transaction. None of it applies: the app connects as postgres, and that role has rolbypassrls = true.

FORCE ROW LEVEL SECURITY does not stop a BYPASSRLS role. FORCE closes the table-owner exemption; BYPASSRLS is a separate and higher one, and policies are not consulted at all. The measured effect: `npm run test:riri` fails four assertions in its RLS section — a freshly-created fixture org reads **515 loans instead of its own 4**, and another org's balance is visible to it. `readpath.ts` already carries the warning in a comment: *"the replica URL MUST be the lms_app role (NOBYPASSRLS) — a replica reached as postgres would serve every tenant's book to every tenant."* The role exists in the design and is not the one in use here.

Confirmed on the live database, 11 Sep 2026. The connection authenticates as the pooler user postgres.<project-ref>, which is the postgres role, and it carries rolbypassrls. This is not a local-environment artefact.

The fix is written and not run: `prisma/rls-app-role.sql` creates the `lms_app` role the design already names, and `npm run db:rls:role` proves isolation against a candidate connection string *before* DATABASE_URL moves onto it. Pre-flight measured the blast radius as small: 61 RLS tables and every one has a policy, the five without RLS are exactly the set `rls.sql` documents as deliberate, and the only two code paths that bypass the tenant stamp both touch `RateLimit`, which has RLS off. Creating a role and rotating the runtime credential on a live book is the founder's to execute.

THE HONEST SUMMARY

The console already contains the best assistant architecture in the estate by a wide margin — a host seam, a pure router with a test, a system map that cannot drift, a bilingual corpus that will not compile if a screen is deleted. **The work is not to design something new. It is to lift that out of one app and make it the thing all six systems and every partner uses** — and to stop the other three from answering at all until they do.

02 Riri Core — one brain, five contracts

The package is extracted from `connected-suite/src/lib/riri` and imports nothing from any application. It never touches a database, a session or a framework. Everything it needs about the world arrives through five typed contracts, and a *host* implements them. That sentence is the whole product:

“add Riri to your system” must not mean “migrate your book to our database first”.

1 • Host

EXISTS – `lib/riri/host.ts`

Who is asking, who they are looking at, what this book contains, what Riri concluded before, and how she writes it down. Already implemented by `providers/lms.ts`. Needs two more: `servicesuite.ts` (over the read-only relay) and `portal.ts` (the customer, one row — their own).

2 • Context

NEW – `lib/riri/context` descriptor

Where she is standing: system, surface, screen, subject, language, rights, entitlements, tenant. **Every field server-resolved.** The client may name an id; it may never state a fact. That rule is already written and enforced in `subject.ts` — this generalises it to the whole descriptor.

3 • Corpus

TWO TIERS, ONE RETRIEVER

Our knowledge is *code* and cannot drift. Their knowledge is *JSON* and cannot describe a screen. §04 is the line between them, and it is what makes “train it on your own data” safe to sell.

4 • Tools

THE ONLY ROUTE TO A FACT

Every live number comes from a tool call. Consent, rights and tenancy are checked *inside the tool body*, never in the prompt — Blueprint v2 §6.2, and the first thing a security reviewer will test.

5 • Destination

NEW – the federated map

Each system publishes a manifest of its screens in the shape `system-map.ts` already uses. Autopilot resolves a sentence to `{ system, href }` across the whole estate, and still stops at the door (§06).

The engines, unchanged

`support • assistant • analytics`

Behind one conversation, picked by the pure router from the finished question, and *stamped on the answer afterwards*. Nobody chooses an engine. Nobody ever should have had to.

What the seam looks like on the wire

A partner does not import TypeScript. They get one signed HTTP contract and a web component, which is the same shape the Interchange already uses for consent — one integration pattern across the whole ecosystem rather than a new one per product.

```
# the only endpoint a partner ever calls
POST https://ai.micro-eazy.africa/v1/ask
  x-riri-org    KE/LENDER/<KRA PIN>      # not EntityId - see §01
  x-riri-ts     1757577600000
  x-riri-sig    HMAC-SHA256 over the exact body

{ "question": "...",
  "context": { "surface":"console", "screen":"/loans/4821",
               "subject": { "kind":"borrower", "id":"..." },
               "lang":"auto" },
  "consent_ref": "cr_..."           # required for anything about a person
}

# the answer always says what it stood on
{ "answer":"...", "engine":"analytics", "evidence":"Queried the live book - SQL attached",
  "sql":"SELECT TOP 50 ...", "actions":[{"kind":"navigate","system":"lms","href":"/console/..."}],
  "sources":[{"pack":"micromart.products","version":7}], "confidence":"certain" }
```

WHY THE SIGNATURE AND NOT AN API KEY

The relay already proves this pattern works across a hostile boundary — HMAC over the exact body inside a two-minute window, in lib/enterprise/relay.ts. Reusing it means a partner integration and a lender's own book are protected by the same reviewed code, and there is one place to fix it rather than two.

03 Context — how she knows where she is standing

“Different contexts, recognise the different screens.” The console already does the hard half: the browser names a subject by id and the server states every fact about them. What is missing is that the *screen* is not yet part of the question, so an officer looking at a loan still has to say which loan.

TABLE 2 · THE CONTEXT DESCRIPTOR — EVERY FIELD RESOLVED SERVER-SIDE

FIELD	WHERE IT COMES FROM	WHAT IT CHANGES
tenant	Session → org. Never the body.	Which corpus bucket, which book, which price book.
actor	Session → name, role title, branch, rights.	The role brief in the persona; what she refuses to describe.
surface	The caller's registered client id.	Console dock · customer app · USSD · partner embed · voice.
screen	The route the caller is on, matched against that system's map.	“What is this?” and “why is this greyed out?” become answerable without naming anything.
subject	An id from the client; facts read from the org-scoped row.	Pins the whole conversation to one person; flips figure questions from the book to the record.
lang	Detected from the question, overridable in Settings.	EN / SW, one corpus with two voices.
consent_ref	Minted by the Consent Center; travels with every call.	No ref, no answer about a person. Checked in the tool, not the prompt.

Screen awareness, concretely

Every screen in every system already declares `purpose`, `does[]`, `implications[]` and `asks[]` in the map. Passing the current route in the descriptor turns that into three things nobody has today:

- “What am I looking at?” answered from the screen's own entry, including the consequences — the half nobody documents and everybody needs.
- “Why can't I do this?” answered from the *difference* between the screen's right/feature and the actor's — so the answer is “your role doesn't hold `loans.approve`, ask your branch manager”, not a walkthrough they will fail halfway through.
- **The proactive line.** On a screen with a known trap, she can open with it. The map already carries the traps; nothing new has to be written.

04 The corpus — ours in code, theirs in JSON

This is the section that makes “put our model in your system and train it with your own JSON” a product rather than a liability. Two tiers, one retriever, and a hard line between them that is enforced by the *shape of the data*, not by a prompt.

Tier A • The platform corpus OURS

CODE • COMPILE-CHECKED • CANNOT DRIFT

`knowledge.ts` articles, `system-map.ts` screens and concepts, `catalog.ts` metrics. An href is verified against the real route table; a right is typed against the real rights catalogue. Delete a screen and the build fails. This is the **only** tier permitted to produce a navigation action.

Tier B • The tenant corpus THEIRS

JSON • EMBEDDED • VERSIONED • REVOCABLE

A lender's products, fees, eligibility, timelines, branch hours, policies — and uploaded documents. Answers what *they* are true about. It can carry an external URL. **It can never carry an in-app destination**, so a tenant pack that is wrong sends somebody to the wrong paragraph, never to a button that does not exist.

One pack schema, everywhere

The Hub's KB files, the C# `SsKbEntry` and the `SeedLenderKnowLedge` body are already three dialects of the same thing. Canonicalise once — this is the file a partner writes, drops in, and is trained on.

```
{
  "pack":      "micromart.products",    // stable id; re-upload replaces this pack only
  "owner":     "KE/LENDER/P051234567X", // KRA PIN, not EntityId (§01)
  "version":   7,
  "audience": ["customer","staff"],    // who may retrieve it
  "lang":      "en",
  "authority": "tenant",                // tenant | platform | document
  "entries": [{
    "category": "products",
    "question": "What loan products do you offer?",
    "variants": ["products gani", "what can I borrow", "nipe products zenu"],
    "answer":   "...",
    "url":       "https://micromart.co.ke/products", // EXTERNAL ONLY – never an app route
    "effective_from": "2026-09-01",
    "review_by":    "2026-12-01"           // an unreviewed fee answer is a wrong fee answer
  }]
}
```

THE RULE THAT KEEPS TIER B SAFE

A tenant pack may state a fact about the tenant. It may never state a fact about the software. Enforced at ingest: an entry whose url is relative, or whose text matches a navigation phrasing, is rejected with a reason. It is also why the answer carries sources[] with the pack and version — when a lender says “Riri told my customer the wrong fee”, the answer is a pack id, a version and a date, not an argument.

Documents are a third authority, not a fourth corpus

The Ratiba agreement, Blueprint v2, the Tailnet Cutover, the operations playbook — these are chunked into the same table with authority: "document" and a page anchor. The difference is in how she is allowed to use them: a document chunk may be *quoted and cited*, never paraphrased into a figure. “The agreement says Safaricom handles activation and transaction-status queries (§6.4)” is a good answer. “Safaricom will fix your Ratiba in 24 hours” is not, and nothing in the document supports it.

TABLE 3 • THE FIRST DOCUMENT PACKS TO INGEST

DOCUMENT	SERVES	ANSWERS IT UNBLOCKS
M-PESA Ratiba API Agreement reports/ • Safaricom, 2024 form	Customers • lenders • API developers	What Ratiba is, *334#, who supports what, what a lender must hold before integrating
The Interchange Blueprint v2 interchange/docs/	Lenders • partners • API developers	Consent scopes, CRB 2.0 report catalogue, tokenisation, reciprocity, the feature taxonomy
Tailnet Cutover reports/ • 11 Sep 2026	Us only NEVER CUSTOMER-FACING	Where a relay lives, why the bureau cannot move, what a funnel does. Internal authority.
Operations Playbook • USSD Analysis • Dormancy & Penalties	Staff	How the floor actually runs, and the figures that are known to be wrong (§10)

05 Live data — tools, not prompts

Blueprint v2 §6 settled the strategy and it is right: the moat is the pool and the tools, not the weights. A frontier model on tools that reach this data beats a fine-tune on any data, and cannot be copied by anyone without the network. What follows is the tool surface, and the one rule that governs all of it.

CONSENT AND SCOPE LIVE IN THE TOOL BODY. ALWAYS.

A prompt instruction is a suggestion to a language model. A check in the tool is a wall. Every tool takes org, actor and, for anything about a person, consent_ref — and validates them before it opens a connection. **No tool in this catalogue writes.** Riri advises; a human presses.

TABLE 4 • THE TOOL CATALOGUE, BY WHO IS ASKING

TOOL	REACHES	STAFF	CUSTOMER	PARTNER
book.metric(id, period, slice)	Catalogue metric on the live book	✓	—	✓ own book
book.query(sql)	Guarded text-to-SQL, SQL always shown	✓ with right	—	✓ own book
customer.record(id)	One borrower's 360, org-scoped	✓	self only	✓
customer.schedule(loanId)	Instalments, due dates, arrears	✓	self only	✓
ratiba.status(msisdn)	Standing order state via Daraja	✓	self only	✓
interchange.exposure(token)	Live multi-lender exposure	✓ consent	—	✓ consent + subscription
interchange.report(token, type)	CRB 2.0 report catalogue	✓ consent	—	✓ consent + subscription
interchange.score(token)	Score + SHAP reason codes	✓ consent	own score, plain words	✓
bureau.metropol(id, type)	Metropol, via salesmaster only	✓ consent	—	—
platform.screen(route)	The system map — purpose, verbs, traps	✓	✓ app routes	✓ own map

Three things to fix in the existing live paths before adding one more tool

- **Replace the substring entity check in** `AiIntelligenceController.RunGuardedQuery` with a parsed predicate or, better, route it through the same catalogue-first path the console uses — a canonical question runs a pre-vetted metric and never reaches a model at all.
- **Give the Interchange node a login of its own.** The suite connects today as `service_suite_user / SystemUser1` — application logins that can write and can read every entity on a shared book. Create `interchange_node` with a view per entity, the `EntityId` baked inside the view, GRANT the

schema and DENY everything else. Until that exists, every tool above is one query-generation bug away from crossing a tenant boundary.

- **Never hand her a bureau tool that is not pinned to salesmaster.** A CRB call from an unregistered egress is dropped at Metropol's edge with no response and nothing in any log — it looks exactly like the bureau being down. The relay now enforces this by egress IP; the tool must not be able to choose a different relay.

06 **Autopilot, across systems**

Today Autopilot works and is well-judged: opt-in, navigation only, resolves a sentence to one of 65 screens, and stops at the door — it will take you to the disbursement screen and it will not press the button, because speech recognition mishears and the gap between “show me” and “send it” is one syllable. The extension is *reach*, not power.

The federated map

Each system publishes a manifest in the shape `system-map.ts` already uses, at a well-known path, with the same drift test guarding it. Riri Core merges them and resolves across the union.

```
GET /.well-known/riri-map.json
{ "system": "lms", "base": "https://console.micro-eazy.africa", "version": 41,
  "screens": [
    { "id": "borrower-new", "href": "/console/borrowers/new", "title": "New Borrower",
      "purpose": "Turn a walk-in into a record from ONE number ...",
      "does": ["Enter a national ID and let IPRS prefill ..."],
      "asks": ["add a borrower", "register a customer", "nisajili mteja"],
      "implications": ["..."], "right": "borrowers.create" }
  ]
}
```

TABLE 5 · WHO PUBLISHES A MAP, AND WHAT IT COSTS THEM

SYSTEM	MAP	WORK
Micro Eazy console	EXISTS 65 screens, drift-tested	Serve it at the well-known path. An afternoon.
Micro Eazy app (customer)	NONE	~14 screens. One file, same shape.
ServiceSuite LMS	PARTIAL Rights/RightsModules already name every controller action	Generate the manifest from the rights table, then hand-write purpose and implications.
Interchange console	NONE	Write it as the screens ship — cheapest when the screen is new.
BirgenAI Hub	PARTIAL app_slug in the KB packs	Promote app_slug to a real map entry.

The three rules that do not move

- **Navigation only.** Autopilot moves the screen. It never submits a form, approves, disburses, sends money or sends a message. Crossing a

system boundary does not change this — it makes it more important, because the user did not see the journey.

- **One destination or none.** `resolveDestination` returns a single screen or refuses. An ambiguous cross-system match is offered as two buttons, never guessed — a wrong jump into another product is far more disorienting than a wrong jump within one.
- **A hand-off is a signed deep link with a reason.** It carries the question that produced it and lands with a dismissible bar: *“Riri brought you here from the console because you asked where Ratiba is set up.”* Somebody who does not recognise where they are must always be able to read why they are there and get back.

07 The customer's Riri — first response

The whole point, stated plainly: **there is no reason for a customer to wait for Geoffrey to be told they press *334#**. Riri answers first, from the tenant corpus and the customer's own record, and only what she genuinely cannot resolve reaches the counter — arriving triaged, with her working shown.

The shape, borrowed from the Vercel agent

The reference screenshot gets one thing exactly right and it is not the styling: the agent *opens by telling you what happens when it fails*. “If we find something I can't solve, I'll help create a support case for you.” That sentence is what makes a customer willing to try the assistant at all, because the cost of it not working is stated up front and it is not “you wasted your afternoon”.

01

Riri answers first, and says so

The Messages composer opens with her line, her scope (“your loan, your repayments, your Ratiba, and how Micro Eazy works”) and the escape hatch. Nothing is created yet — a question that resolves never becomes a thread, and never becomes a number on Geoffrey's counter.

02

She reads before she answers

Tenant corpus for “what do you charge”. Her own record tools for “when is my next instalment” — `customer.schedule`, `ratiba.status`, `customer.record`, all self-scoped by the proven phone in `borrowerFor()`. The customer is never trusted for identity; that is already true of the portal and does not change.

03

Three outcomes, and only one of them is a ticket

Resolved — answered, thumbs-up/down, logged as a deflection.
Resolved with an offer — answered, plus “want me to raise this with your officer anyway?”.
Escalated — she opens the thread herself.

04

An escalation arrives with the triage already done

She writes the first message on the thread, in her own voice, addressed to the officer: what the customer asked, in their words; what she checked and what it said; which KB packs she matched and what she did *not* find; and the one thing she needs a human to decide.

`findOrOpenThread` and `postMessage` already exist — the kind, the subject and the `applicationId` are hers to set, and the ownership check on `applicationId` stays exactly where it is.

05

The counter learns from what it stops seeing

Every deflection is a row: question, matched packs, confidence, outcome. A weekly view of “the ten questions Riri answered most” is the lender's own product backlog. A view of “the ten she escalated most” is the next ten KB entries somebody should write — and after they are written, the escalation rate for that cluster should fall. That is the only honest measure of whether any of this works.

WHAT SHE MUST NEVER DO ON THIS SURFACE

- State a figure she did not read from the record in this conversation. No estimates, ever.
- Promise an outcome, a timeline or an approval. She may describe the process; she may not commit the lender to it.
- Tell a customer their payment will go somewhere it will not. Micromart's `RepaymentTrigger` allocates to the loan first and the remainder to savings — *a customer cannot choose*, and an assistant that implies otherwise creates a dispute over real money.
- Read back a full phone number or an ID number unless asked for it specifically.
- Keep going when the customer asks for a person. “Let me get you to someone” is an instant, cheerful escalation — an assistant that argues about being replaced is the thing everyone hates about assistants.

08 Ratiba from the horse's mouth — and the agreement it implies

The Safaricom agreement is worth ingesting for a reason bigger than Ratiba: it is a *working model* of how to let somebody into a payment network — what the network hosts, what the joiner must hold, who answers which support call, and what happens when they stop. That is exactly the document the ecosystem is missing for its own lenders.

What the agreement actually establishes

TABLE 6 · RATIBA — FACTS FOR THE CORPUS, TAKEN FROM THE AGREEMENT

FACT	WHO IT ANSWERS FOR
Ratiba is M-PESA's standing-order feature, reached on *334# and in the M-PESA app; customers create subscriptions, set recurring payments and get due-date notifications.	Customer, instantly, forever
Safaricom hosts the API Solution and bears development, operationalisation and marketing costs.	Lender, API developer
Safaricom handles customer queries about transaction status, M-PESA statements, and Ratiba activation/deactivation. Everything about the service relationship is the Company's.	Support routing — this is the escalation boundary
The Company integrates directly, procures its own paybill / M-PESA accounts, and carries its own legal and tax compliance.	Lender onboarding checklist
Non-exclusive. 12-month initial term, auto-renewing, 90 days' notice not to renew.	Commercial
Use is subject to the M-PESA Ratiba T&Cs and the Lipa na M-PESA T&Cs, incorporated by reference.	Compliance

RATIBA IS ALREADY BUILT HERE

lib/mpesa/daraja.ts, /api/portal/standing-order, the onboarding Ratiba.tsx step and AutoRepayCard all exist. This section adds no code to the rail — it adds the *knowledge*, so the question “do I need to press *334#?” is answered in under a second by the same system that already set the mandate up.

The Micro Eazy Ecosystem Participation Agreement

Built on the Safaricom skeleton, issued as a step in organisation creation, and stored against the org the way the onboarding contract already is. Blueprint v2 makes consent the spine of the network and says the integration burden of joining is deliberately the first thing a new member builds — this is the paper half of that, and the signature is what turns on their first query.

TABLE 7 · CLAUSE MAP — WHAT WE BORROW, WHAT IS OURS

CLAUSE	SOURCE	SUBSTANCE
Scope	Safaricom §3	Non-exclusive. The Member integrates to the Interchange and the Micro Eazy rails; the Member procures its own paybill, licences and CBK/DPA standing.
What we host	Safaricom §4	Nodes, Registry, Consent Center, the AI layer, the relays. The Member gets an endpoint, not a server (Blueprint v2, Table 1).
Consent	Blueprint §2	The eight scopes, verbatim. No consent_ref, no answer. Revocation propagates within one config cycle.
Reciprocity	Blueprint §1	Contribution is the price of querying, enforced in the policy engine and not merely promised here. SHADOW until first ingest commits.
Support boundary	Safaricom §4.4	We answer platform, interchange and AI questions. The Member answers everything about their own lending relationship. Written down, because Riri routes on it.
The write boundary	Ours	The relay is read-only. Named write grants only, per-org, per-procedure, per-parameter, listed in the agreement's schedule so a Member can see every door that exists.
AI use of Member data	Ours	Tool-gated retrieval always. Training on the Member's book <i>only</i> under model.train consent, de-identified past the tokenisation boundary.
Data protection	Safaricom §8 + DPA	Kenya Data Protection Act, processor/controller roles named, breach notification, sub-processor list.
Schedules	Safaricom form	1 · Features. 2 · Policies. 3 · Consent wording (canonical). 4 · Write grants. 5 · Fees and the free tier.

Delivery: generated as a PDF from the org's own particulars at /console/settings/organisation, signed in-app, stored beside the onboarding contract, and re-issued on version change. The unsigned state is a real gate — an org without a current signed agreement can configure everything and query nothing, which is the same shape as activation today.

09 Where it runs

The Tailnet cutover gives the AI layer its home without any new hardware. Two funnels on two nodes, and the reason they cannot be one is the same reason Riri's bureau tool must be pinned: a funnel changes where a relay can be *reached*, never where it *calls from*.

TABLE 8 · THE AI LAYER ON THE EXISTING ESTATE

NODE	CARRIES	RIRI'S DEPENDENCE
servicesuite-in 100.115.241.36 : 8443	SQL relay + the Interchange node	PRIMARY every lender-book tool terminates here. Also where the OPRF tokenisation runs, so tokens are minted on the same box and the same route.
salesmaster 102.214.69.233 : 443	CRB relay, and nothing else	PINNED the only permitted origin for bureau.metropol.lms is the standby that would actually work.
Vercel	Console, portal, Riri Core orchestration	Holds no database route. Signs, calls tools, streams.
Supabase pgvector	riri_knowledge — one project, scoped rows	Tier B and documents. Tier A stays in git.

TWO STANDING RULES THIS PLAN INHERITS

Never disrupt Micromart's shared server. Everything the AI layer adds is additive — a service on a port, a funnel mapping, a login with a view per entity. No iisreset, and check what shares an app pool before touching one.

Never type a bare tailscale funnel off on these nodes: on salesmaster it silently kills the bureau link, on servicesuite it takes the live SQL path down and all six systems go amber at once. Always name the port.

One prerequisite worth stating on its own: the tailnet policy in relay-host/tailnet-policy.hujson is unapplied, and until it is, eight nodes hold a standing route to two lenders' live SQL Servers — which means Riri's careful tool boundary is bypassable by anyone who can open a TDS socket directly. Apply it in its own change window, after the cutover is green.

10 The grounding traps

A retrieval system is only as honest as the facts it is pointed at, and this estate has several places where the obvious reading of the data is wrong. Each of these becomes a pinned corpus entry with authority: "platform" before she is allowed to answer in that area at all — because the failure mode is not silence, it is a confident, plausible, wrong number.

TABLE 9 · KNOWN-WRONG READINGS, AND WHAT SHE MUST SAY INSTEAD

AREA	THE TRAP	THE PINNED TRUTH
Settlement dates	Loans.DateCleared is NULL on all 344,332 rows	Derive settlement from CustomerStatement. Any metric built on DateCleared must be refused, not returned as zero.
Penalties	Reads as a live penalty regime	Effectively off — 6 of 37 products, once per loan, dormant since June, and a NULL bug eats about a third of the figures. Say that, with the caveat, or say nothing.
KYC results	The OCR reports 96% confidence and a full identity	It <i>simulates</i> when ocr.engine says so. A simulated result must never be stated as a verification outcome to anyone, staff or customer.
Repayment allocation	A “what is this payment for?” picker looks like routing	RepaymentTrigger owns it: loan first, remainder to savings. The customer does not choose.
SMS	The obvious table is not the live one	Live SMS is Notifications.dbo.SMS. 3002 and 3005 are one database.
Member identity	EntityId 3003 names two different lenders on two servers	Key on the KRA PIN (BsEntity.EntityTaxRef). Never answer a cross-server question keyed on EntityId alone.

These are not edge cases, they are the six questions a lender is most likely to ask in their first week. Writing them down is cheap; discovering them through a wrong answer in front of a customer is not.

11 The build

Ordered so that each sprint ships something usable on its own, and so the riskiest integration — a partner's system — comes after the contract has been proven twice inside our own estate. Nothing here needs new hardware and only one item needs a decision.

TABLE 10 · SPRINTS

#	WINDOW	DELIVERABLE	DONE WHEN
0	BUILT 11 SEP	The scope guard, one reviewed implementation for both text-to-SQL call sites, with 30 pinned cases. The embedding model pinned in one module. The pack schema and its ingest rules. The six grounding traps, wired into the live answer path. The Ratiba pack. Riri renamed, with the white-label seam.	Done — 30/30 guard, 73/73 knowledge, 68/68 router. Two items remain: the Supabase re-seed (a live write, yours to trigger) and the Interchange document pack.
1	Weeks 1–2	Extract @riri/core from connected-suite/src/lib/riri — the five contracts, no app imports. Console keeps working through providers/lms.ts unchanged.	The console is a <i>host</i> , not the owner. Router and map tests still green.
2	Weeks 2–4	Riri First Response in the customer app: the composer, the three outcomes, the triaged escalation, deflection logging.	A customer asks about *334# and is answered without a thread being opened.
3	Weeks 4–6	The context descriptor and screen awareness. Federated map manifests for console and app. Cross-system Autopilot with the signed deep link and the “why you are here” bar.	“Take me to add a borrower” works from the customer app, the console and a partner embed.
4	Weeks 6–8	The tenant training console: upload a pack, validate it, see what it will answer, version it, roll it back. The ingest-time rejection rules from \$04.	A lender trains Riri on their own JSON without us touching anything.
5	Weeks 8–11	providers/servicesuite.ts over the relay + the Interchange tools behind consent_ref. The Participation Agreement in org creation.	A loan officer asks a question only this network can answer, and the agreement that permitted it is on file.
6	Months 3–4	Cut the C# AiIntelligence module over to the HTTP contract and retire the duplicate RAG. Publish the partner embed SDK and the web component.	One Riri. Four surfaces retired into one brain, and an outside system can host her.

START HERE, AND IT NEEDS NO DECISION FROM ANYONE

Sprint 0 is four small tasks and every one of them is a correctness fix to something already live. The substring entity check in particular should not survive the week: ten unrelated companies share the .198 book separated by nothing but EntityId, and the guard that is meant to keep them apart currently passes on a coincidence in a date.

12 Decided, and what is left

SETTLED 11 SEPTEMBER 2026

- **She is Riri**, white-labelled per host. `brand.ts` now exports "Riri" plus an `assistantName(host)` seam; the ~40 call sites that read the constant flipped with it, and the ten hard-coded strings in the metric console now read it too. Internal identifiers did not move.
- **Her voice is ours, the facts are theirs, and the pack id is on every answer.** `sourceRef()` in `pack.ts` is that decision in code.
- **A partner's Riri sees their own book and the Interchange under consent** — not our system map, our metric definitions, or another member's packs. Enforced by the `pack owner/audience` fields and to be written into the agreement's schedule.
- **Escalations change shape, and that is a deliberate choice** rather than something to discover in a month. Revisit at Sprint 2, when the deflection log has real rows.

The four below are what the decisions opened up rather than closed.

01

Is production connecting as a NOBYPASSRLS role?

ANSWER FIRST

The \$01 finding is measured in this environment only. If Vercel's `DATABASE_URL` also carries `BYPASSRLS`, tenant isolation on the live console is decoration. It is one query to settle and it outranks everything else on this page.

02

When to run the re-seed

The embedding path is pinned but the existing rows were written before it was. They are in the right space, so this is hygiene rather than a fix — worth doing in a quiet hour, with the seeder's new boot line captured, so there is a record of which model the corpus stands in.

03

One name, or two

DONE

The console calls her `ServiceSuite AI`; the Hub and the C# module call her Riri. If she is to be sold into a competitor's console, the outward name should be brand-driven per host — one string, already isolated in `brand.ts` — while the persona stays constant. My recommendation: **Riri is the product name, white-labelled per host**, and the internal identifiers never change.

04

Whether the customer's Riri is the lender's or ours

A Micromart customer asking about a Micromart fee is answered from Micromart's pack. But whose voice apologises when she gets it wrong, and whose name is on the deflection report? My recommendation: **her voice is ours, the facts are theirs, and the pack id is on every answer** — which is also the only version that survives a dispute.

05

How much of the estate a partner's Riri may see

A lender hosting Riri gets their own book and the Interchange under consent. They do not get the system map of our console, our metric definitions or another member's packs. Worth writing into the agreement's schedule rather than leaving to the tool layer to imply.

06

Whether escalations reach Geoffrey, or a queue

Riri makes the counter smaller and sharper. It also changes what arrives: fewer, harder, pre-triaged. That is a different job from answering fifty “when is my due date” messages a day, and it is worth deciding deliberately rather than discovering it in a month.

WHAT IS BUILT, AND WHAT WAS DELIBERATELY LEFT

Sprint 0 is in the working tree of three repositories and is described in Table 10. Nothing was deployed, no relay was touched, and no query was run against any lender's live book. The two reads that did happen were against this console's own Postgres, were read-only, and were catalogue queries run to diagnose the RLS finding in §01.

Two Sprint 0 items are deliberately not done, because both are outward-facing and yours to trigger. The Supabase re-seed rewrites every row in a table the live Hub reads, so the code is pinned and the seeder now prints its model at boot, but it has not been run. And the six decisions the founder made on 11 Sep are implemented in code — Riri as the product name with a white-label seam, the pack id on every answer — while the Participation Agreement they imply (§08) is specified here and not yet drafted.